C++ cascading is multiple use of input and output operators (>> and <<) in one statement

Dt.: Pg.: Delta

C++ was developed by Bjarne Stroustrup at Bell labs. C++ is an intermediate level language as it comprises of high level and low level language.

C++ is an Object Oriented language (OOPs) but is not a purely object oriented. Its features like Friend and Virtual violate some very important OOPs features.

Benefits of C++ over C language

(i) Stronger type checking in C++.

(ii) features like Abstraction, Encapsulation, Inheritance etc makes its useful

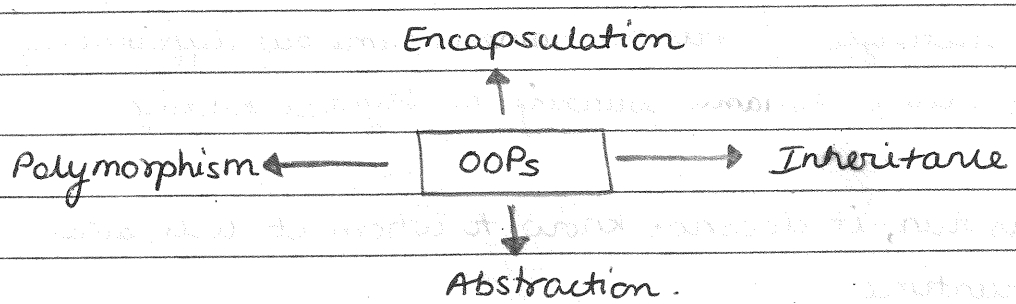
(iii) allows user defined operators (i.e. Operator Overloading) and function overloading.

(iv) Exception Handling is there.

(v) concept of virtual functions and constructors, destructors for objects.

(vi) Inline functions in C++ instead of Macros in C.

(vii) Variables can be declared anywhere in program but must be declared before they are used.



Object Oriented Programming is a style that is associated with concept of class, object and various other concept revolving around these two, like inheritance, abstraction, etc.

Dt.:
Pg.:

class

A class is a blue print for any functional entity which defines its properties and its functions.

Inheritance

child can derive features and attributes of parents. Inheritance is a way to reuse once written code ajain and ajain.

objects

every entity is an object. objects are basic unit of OOP. They are instances of class.

Abstraction

internal details are hidden from user, only necessary details are shown. This can be done using access specifiers. (Private, protected etc)

Encapsulation

binding of properties to functions is called Encapsulation.

Polymorphism

function name is same but in outerworld it can be used differently.

ex: shape()

circle triangle rectangle. | Function name is same but different arguments

→ also called dynamic bounding or layered binding.

Dynamic binding

when procedure is run, it doesnot know to whom it will bind, which is done at runtime.

Dt.:
Pg.:

Data types in C++.

They are used to define type of variables and contents used.

Built in data types.

They are data types which are predefined and are wired directly into compiler eg: int, char etc.

User defined or Abstract data types.

These are type, that user creates as a class. In C++ there are classes.

→ basic built in types

char (1 byte), int (2 byte), float (4 byte), double (8 byte), bool (True or false), void (without any value), wchar_t (wide character)

modifiers.

- long
- short
- signed (includes both positive and negative no. and is default type)
- unsigned. (numbers are always without any sign, and are +ve).

Scope of variables.

all variables have their area of functioning, and out of that boundary they don't hold their value, this boundary is called scope of variables. ← Global variable
Local variable.

Global variables - are those which are once declared and can be used throughout lifetime of program by any class or func^n. They must be declared outside the main() func^n.

Local variables - variables which only exist b/w curly braces, in which its declared. outside that they are unavailable and leads to compile time error.

Dt.:
Pg.:

Operators in C++.

Operators are special type of functions, that takes one or more arguments and produce a new value.

Assignment operator (=), is used for assignment and is the only operator which can be overloaded but cannot be inherited.

Mathematical operator (+, -, /, *, %) is used to perform mathematical operations. Modulus operator (%) cannot be used with floating-point number.

Relational operator (<, >, <=, >=, ==, !=) are used to establish relationship b/w operands.

Logical operator (&&, ||) are used to combine two different expression together. mostly used in loops and in Decision making.

Bitwise operator (&, |, ^(XOR), ~(NOT))
used to charge individual bit into a number. They work only with data types like char, int, long and not with floating point values.

Shift operator (<< (left shift), >> (right shift), >>> (unsigned right shift)) are used to shift bits of any variable.

Unary operator are operators which work only on one operand. (++, --). other unary operator are address of (&), derefrence (*), new and delete, bitwise not (~), logical not (!), unary minus (-) and unary plus (+).

Ternary operator also called conditional operator. (?:) is an operator which has three operands.

```
a>5 ? cout<<"true" : cout<<"false"
```

true

false

Comma operator is used to seperate variable names and to seperate expressions. In case of expressions, last expression value is produced and used.

eg: a = b++, c++; // a = c++ will be done.

sizeof operator is an operator and not a function. and is used to get information about amount of memory allocated for data types & objects. It can be used to get size of user defined data types too.

eg: sizeof (double) or int x = 2; sizeof x;

typedef operator is a keyword used in C language to assign alternating names to existing types.

eg: typedef unsigned long ulong;

Now you can use ulong as an identifier.

ulong i, j;

Can also be used to give alternative name to pointers also.

eg: int *x, y; we are declaring x as a pointer and y as a variable.

typedef int * Intptr;

Intptr x, y, z; // you can declare any no. of pointers in single statement.

Decision making.

if / if else | switch | conditional operator

goto statement

Looping in C++.

while | for | do-while

Jumping out of loop.

└ break: loop is immediately exited and program continues with statement immediately following the loop

└ continue: it causes control to go directly to test condition and then continue the loop process.

Dt.:
Pg.:

Storage classes in C++.

are used to specify the lifetime and scope of variables. How storage is allocated for variables and how is variables treated by compiler depends on these storage classes.

Global variables are defined at starting, before all functions and are available throughout program.

Local variables are defined and are available within a particular scope. They are also called automatic variables because they come into being when scope is entered and automatically go away when scope ends.

Register variables is a type of local variable. This keyword is used to tell the compiler to make access to this variable as fast as possible. You cannot have global or static register variable.

static variables are variables which are initialized & allocated storage only once at beginning of program execution, no matter how many times they are used and called in program. A static variable retains its value until the end of program.

```
Void func()
{   static int i = 10;
    i++;
    cout <<i;
}
int main()
{
  func();        // output = 11;
  func();        // output 12;
  func();        // output = 13;
}
```

Extern variables : is used to access variables in a file which is declared & defined in some other file, that is existence of global variables in one file is declared using extern keyword in another file.

File1.cpp

```
#include <iostream.h>
using namespace std;
int globe;
void func();
int main()
{
  .....
}
```

File2.cpp

```
extern int globe;
int b = globe + 10;
{ Global variable in one file is used in other file by extern keyword }
```

Functions

```
#include <iostream.h>
using namespace std;
int sum(int x, int y);          // declaring funcn
int main()
{
    int a = 10, b = 20;
    int c = sum(a,b);           // calling funcn
    cout << c;
}
int sum (int x, int y)          // defining funcn.
{
    return (x+y);
}
```

Dt.:
Pg.:

Calling a function.

functions are called by their names. If function is without argument, it can be called directly by using name. But for function with arguments, we can call by two ways.

Call by Value.

- we pass the values of argument which are stored or copied into formal parameters of functions. Hence, the original values are uncharged only the parameters inside func^n changes.

```
void calc (int x);
int main ()
{
    int x = 10;
    calc (x);
    printf ("%d", x);          // output is 10.
}                              actual value of x is not
void calc (int x)              changed, copied value is
{                              destroyed because value
    x = x + 10;                was not returned.
}
```

call by reference

- in this we pass the address of variables as arguments.
- they changes values of original ~~value~~ variables.

```
int main ()
{                                  void calc ( int *p)
    int x = 10;                    {                          //output=20
    calc (&x);                       *p = *p + 10;
    printf ("%d", x);              }
}
```

Dt.: Pg.: Delta

Introduction to classes and objects

Class is a user defined data type, which holds its own data members and member functions, which can be accessed and used by creating instances of that class.

The variables inside class definition are called data members and functions are called member function:

(i) class name must start with an uppercase letter.

(ii) class defaults to private access control, structure defaults to public

(iii) Objects can hold seperate copies of data members. We can create as many objects of class we need.

Object are instances of class, which hold data variables declared in class and member functions works on there class objects. Each object has different data variables.

Objects are initailized using special class functions called Constructors. And wherever the object is out of scope, another special class member function called Destructor is called to release reserved memory by object.

C++ doesnot have Automatic Garbage Collector like in Java, in C++ Destructor performs this task.

Access Control in classes

access specifiers in C++ class defines the access control rules. These access specifiers are used to set boundaries for availability of members of class be it data members or member functions.

Public: means all class members declared under public will be available to everyone.

Private: means no one can access the class members outside that class. If someone tries to access these, they will get compile time error. By default, class variables and member functions are private.

Protected: it makes class members inaccessible outside the class. But they can be accessed by any subclass of that class.

Accessing data members of class. (. operator)

accessing data member depends on access control (public, private, protected) of that data member.

If it is public, data member can be easily accessed using direct access (.) operator with object of that class.

Defining func^n outside class (:: operator)

```
class cube
{
    public:
    int side;
    int getVolume();
};
int Cube :: getVolume()          //defining outside class
{
 return side * side * side;
}
```

Types of Member function

(i) simple function: These are basic member func^n which donot have any special keyword as prefix.

```
eg:  int cube(int side)
     {
        return side * side * side;
     }
```

(ii) Static function: is something that hold its position. static is a keyword which can be used with data members as well as member func^n.

Dt.: Pg.: Delta

It can be called using the object and direct access member (.) operator. But, its more typical to call a static member function by itself, using class name and scope resolution (::) operator.

```
class ABC
{ static int count;
  public :
  static void f() -
   { };
 };
int main()
 {
   ABC ::f() ;          // calling member func^n directly with class name
 }
```

if ~~three~~ there are 3 objects of class, then value of count will be common for all. values changes made by object 1 will be same for object 2

* These func^ cannot access ordinary data members and member functions, but only static data members and static member functions.

* static data member variable should always be defined outside the class also.

example:

```
class test
{
    int no ;
    static int count;                    // static data variable
    public :
        void setno (void);
        void dispno (void);
        static void dispcount (void);    // static func^n.
};
```

```
void test :: setno(void)
  {
    no = ++count;
  }

Void test :: dispno (void)
  {
    cout << " object no" << no;
  }

static void test :: dispcount (void)
  {
    cout << " counter" << count;
  }

int test :: count;          // defining, outside class also
                            static variable

int main()
{
  test t1, t2;
  t1.setno();               // calling normal func^n.
  t2.setno();
  test :: dispcount();      // calling static func^n
  test t3;
  t3.setno();
  test::dispcount();
  t1.dispno();
  t2.dispno();
  t3.dispno();
}
```

Dt.: Pg.: Delta

(iii) constant func^n.

- constant keyword makes variable constant, that means, once defined there values can't be changed.
- when used with member func^n, such member func^n can never modify the object or its related data members.

Syntax:

```
void func() const
{
}
```

ex:
```
add (const int x)
{
  x = x+10;
}
```
✗ x value cannot be changed

```
add (const int x)
{
  y = x+10;
}
```
✓ x value is not changing

```
add(const int & x);
add(const int *p);
```
} can be written.

→ default argument

You cannot pass arguments from left to right.

```
(int x, int y, int z = 10)   ✓
(int x, int y = 5, int z = 10)   ✓
(int x = 10, int y, int z)   ✗
```

```
int add (int x, int y, int z = 10, int s = 100)
{ }
```
both add will be executed.

```
add (10, 20);
add (10, 20, 30);
```
→ this z will be executed

Dt.:
Pg.:

(IV) Inline func^n

All member functions defined inside the class definition are by default declared as Inline.

Inline functions are actual functions, which are copied everywhere during compilation, so overhead of function calling is reduced.

You can also make any non-class function inline by using keyword inline with them.

For an inline function, declaration and definition must be done together.

```
# include <iostream.h>
inline int max(int x, int y)
  {
    return ( (x>y) ? x : y );
  }
int main()
 { int a, b;
   cin >> a >> b;
   max(a, b);
 }
```

```
class S
{
 public:
   int square(int s);
};

inline int S::square(int s)
{
}
```

Important Points

1) We must keep inline func^n small.
2) We should not make large func^n inline bcz it may lead to code bloat and might affect speed too. ie. is why large func^n are defined outside class using (::) operator bcz if declared inside class, then they become inline by default
3) Inline func^n are kept in Symbol Table by compiler, and all the call for such func^n is taken care at compile time.

Dt.: Pg.: Delta

(v) Friend function

are actually not class member function. Friend func^n are made to give private access to non-class func^n. You can declare a global func^n as friend, or a member func^n of other class as friend.

Here, friend func^n can access private data members by creating objects of class.

When we make a class as friend, all its member func^n automatically becomes friend functions.

Friend func^n is a reason, why C++ is not called a pure object oriented language.

(i) (::) should not be used bcz belong to both class

(ii) call disp() → call without any object as argument

(iii) friend void disp (class object)

↑ argument is class object.

```
# include <iostream.h>
class A ;
class B
 {
    int b ;
    public :
        void get_val( int x)                    void  put_val (void)
         {                                      {
            b = x;                                  cout << b ;
         }                  → class name }
    friend  void add ( A, B );
} ;
```

```
class A
{
    int a;
    public :
        void get_val (int x)
        {
            a = x;
        }
        void put_val ( void)
        {
            cout << a;
        }
    friend void add (A, B);
};

void add ( A obj1 , B obj2 )
{
    cout << obj1.a + obj2.b;
}

int main()
{
    A a;
    B b;
    a.get_val(100);
    b.get_val (200);
    a.put_val();
    b.put_val();
    add (a, b);
}
```

↳ objects are passed to friend func^n.

Dt.: Pg.: Delta

→ class and object. (definition given before)

```
class class_name
{
  private:
      data declaration;  func^n declaration;
  public:
      data declaration;  func^n declaration;
};
```

```
ex:  class person //class      person p; //object
     {
     };
```

by default, if you don't write anything, class member and func^n are private.

private : can be used only in that class.
public : can be used outside class.

example :

```
class Student
{
  private:
     int roll;
     char name[10];
  public:
     void getdata();
     void putdata();
};
```

```
student st;
student st1, st2;
```
} only public func^n can be accessed

```
st.getdata(); → now private data can be accessed.
st.roll; → this is wrong, object cannot directly access private data, can access only by public func^n.
```

Dt.:
Pg.:

Delta

Access specifiers.

(i) private : can be used only in class, not outside class.

(ii) protected : used during inheritence, can be used by class and derived class.

(iii) public: can be used by all public func^, can only access private data and member.

example:

```
class test
{
  int roll;
  float per ;        // percentage
  public:
      void getdata (int, float) ;
      void putdata( )                    // inside declaration
      {
          cout << "roll = " << roll;
          cout << "per = " << per;
      }
};

void test:: getdata (int x, float y)  // outside declaration
  {
    roll = x;
    per = y;
  }
main
{  test t;                    // object
   t.getdata ( 101, 80.3 );    // func^ call
} t. putdata ();
```

Dt.:
Pg.:

→ Scope resolution → unary (if there is global & local variable with same name, to access global variable, use unary operator)

Eg: sum → is a global variable
→ is a local variable also.

```
cout << sum;        print local variable
cout << :: sum;     print global variable.
```

Dt.:
Pg.:

Variations in for... loop

(I) Multiple initialization and update expression

```
for(i=0, sum=0 ; i<=n; sum++, i++)
```

(II) optional expression

```
for( ; i<=n; )
```

(III) Infinite loop

```
for( ; ; )          //this will not cause error
```

(IV) Empty loop

```
for(int i=1; i<=300; i++);
```

Variable declared inside for loop can't be accessed outside loop.

Dt.: Pg.: Delta

Function Overloading

If any class have multiple functions with same names but different arguments then they are said to be overloaded. Function overloading allows you to use same name for different functions

If you want to perform one single operation but with different no. of/types of arguments, then you can simply overload the func^n.

```
sum ( int x, int y = 0);          ✓
sum ( int x, int y = 0 ) int z);  X  This is incorrect
sum ( int x, int y = 0, int z = 0); ✓
```

Place holder arguments.

When arguments in a func^n are declared without any identifier they are called placeholder arguments.

```
Void sum ( int, int );      ✓
Void sum ( int, int = 0);   ✓
```

```
#include <iostream.h>
int calc( int x)
    { return x * x; }
int calc (int x, int y)
    { return x * y; }

int main()
 { int a, b;
   cin >> a >> b;
   cout << "area of square" << calc(a);
   cout << "area of rectangle" << calc(a,b);
 }
```

Dt.:
Pg.:

Constructors

Constructors are special class func^n which performs initialization of every object. The compiler calls the Constructor wherever an object is created. Constructors initialize values to object members after storage is allocated to object.

While defining constructor, you must remember that the name of constructor will be same as name of class, and constructor never have return type.

```
class A
{
  int i;
  public:
    A();                          // constructor declaration
};

A::A()                            // constructor definition
{
    i = 1;
}
```

(1) Default constructor is constructor which doesn't take any argument. It has no arguments. In this case, as soon as object is created the constructor is called which initializes its data members.

If we donot' define a constructor explicitly, the compiler will provide a default constructor implicitly and initialize to 0.

- above example is default constructor.

Dt.:
Pg.:

(ii) Parameterized constructor - with parameters. Using this constructor, you can provide different values to data members of different objects, by passing appropriate values as argument.

```
class cube
{
  int side;
  public:
        cube (int x)                    // parameterized constructor
        {
          side = x;
        }
};
int main()
{
  cube c1(10);                          } → giving different values to
  cube c2(20);                            constructor.
  cout << c1.side;
  cout << c2.side;
}
```

(iii) Copy constructor is a special type of constructor which takes an object as argument, and is used to copy values of data members of one object into other object.

It is usually of form X(X&), where X is class name. Compiler provides a default copy constructor to all classes. It creates a new object, which is exact copy of existing copy.

Syntax:

```
classname (class_name &)
{ ...
}
```

Dt.:
Pg.:

Constructor Overloading

Constructors can be overloaded. Infact when you have both default and parameterized constructors defined in your class you are having Overloaded constructors. You can have any no. of constructors in a class that differ in parameter list

Note: if you have defined any constructor explicitly, then the compiler will not provide default constructor and you have to define it yourself.

```
class student
{
  int rollno;
  string name;
  public:
      student (int x)
      {
        rollno = x;
        name = "None";
      }
     student (int x, string str)
      {
        rollno = x;
        name = str;
      }
};
int main()
{
    student A(10);
    student B(11, "Ram");
}
```

Important points

(i) public
(ii) no return value
(iii) inheritence not possible
(iv) virtual
(v) cannot be suffered by address
(vi) default argument can be used.

It will lead to compile time error, bcz we haven't defined default constructor and compiler will not provide its default constructor bcz we have defined parameterized constructors